

# **AI-Powered UGC Ads Video & Image Generator**

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF SCIENCE**

in

**COMPUTER SCIENCE**

BY

**Kratika Gupta (SC23CS302016)**

**Kritik Pandey (SC23CS302017)**

Under the Guidance of

**Dr. Rohit Gupta**

**Dr. Shrikant Telang**



**Department of Computer Science**

**Faculty of Science**

**MEDICAPS UNIVERSITY, INDORE- 453331**

**APRIL-2026**

# Report Approval

The project work “**AI-Powered UGC Ads Video & Image Generator**” is hereby approved as a creditable study of a science subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

# Declaration

We hereby declare that the project entitled “**AI-Powered UGC Ads Video & Image Generator**” submitted in partial fulfillment for the award of the degree of Bachelor of Science in ‘Computer Science’ completed under the supervision of **Dr. Rohit Gupta and Dr. Shrikant Telang**, Department of Computer Science & Engineering, Faculty of Engineering, Mediacaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

**Signature:** \_\_\_\_\_

**Name:** Kratika Gupta (SC23CS302016)

**Date:** April 2026

**Signature:** \_\_\_\_\_

**Name:** Kritik Pandey (SC23CS302017)

**Date:** April 2026

# Certificate

We, **Dr. Rohit Gupta and Dr. Shrikant Telang** certify that the project entitled “**AI-Powered UGC Ads Video & Image Generator**” submitted in partial fulfillment for the award of the degree of Bachelor of Science by **Kratika Gupta and Kritik Pandey** is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

---

**Dr. Rohit Gupta**

**Dr. Shrikant Telang**

Department of Computer Science & Engineering

Medicaps University, Indore

---

**Prof. (Dr.) Kailash Chandra Bandhu**

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

## Acknowledgements

We would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Mediacaps University, whose unfailing support and enthusiasm have always boosted up my morale. Furthermore, we thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean of Engineering, Mediacaps University, for giving me a chance to work on this project. We would also like to thank our Head of the Department **Prof. (Dr.) Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

We sincerely thank our project guides, **Dr. Rohit Gupta, Dr. Shrikant Telang**, for the expert guidance, valuable suggestions, and continued motivation throughout the development of this project. The timely feedback and technical advice provided helped me navigate through challenges and deliver a quality outcome.

We would also like to express our gratitude to all the faculty members of the Department of Computer Science & Engineering, Mediacaps University, for their direct or indirect support and academic inspiration. Special thanks to our fellow students and classmates for their cooperation, ideas, and moral support throughout this project.

Finally, we thank our family for their unconditional support and encouragement throughout my academic journey. This project would not have been possible without their love and patience.

**Kratika Gupta (SC23CS302016)**

**Kritik Pandey (SC23CS302017)**

B.Sc. III Year (A)

Department of Computer Science

Faculty of Science

Mediacaps University, Indore

# Abstract

In the modern digital era, businesses and content creators require engaging, creative, and personalized advertisements to attract and retain customers. However, traditional advertisement creation is time-consuming, requires expertise, and lacks scalability. To address these challenges, this project **titled “AI Powered UGC Ads Video & Image Generator”** presents an intelligent solution that automates the generation of user-generated content (UGC) advertisements using artificial intelligence.

The system is designed as a full-stack web application built using the PERN stack (PostgreSQL, Express.js, React.js, and Node.js). It enables users to generate advertisement content such as captions, product descriptions, reviews, scripts, images, and videos by simply providing a text prompt or uploading media. The application integrates with the Google Gemini API to produce high-quality, context-aware, and creative outputs in real-time.

The platform includes secure user authentication using Clerk, ensuring that only authorized users can access the system. It also provides features such as content history management, allowing users to view, reuse, and manage their previously generated advertisements through a personalized dashboard. The backend system efficiently handles API communication, input validation, error handling, and database operations, while the frontend provides a responsive and user-friendly interface.

Furthermore, the system ensures performance efficiency by generating outputs within a short time and maintaining data integrity using PostgreSQL. The modular architecture of the application enhances scalability, maintainability, and future extensibility.

This project significantly reduces the effort and time required for advertisement creation, improves productivity, and enables users with minimal technical knowledge to generate professional-quality marketing content. It demonstrates the effective integration of artificial intelligence with modern web technologies and highlights the growing importance of AI-driven solutions in digital marketing and content generation

# Table of Contents

| Chapter No. | Topics                     | Page No. |
|-------------|----------------------------|----------|
|             | Report Approval            | ii       |
|             | Declaration                | iii      |
|             | Certificate                | iv       |
|             | Acknowledgement            | v        |
|             | Abstract                   | vi       |
|             | Table of Contents          | vii-ix   |
|             | List of figures            | x        |
|             | List of tables             | xi       |
|             | Abbreviations              | xii      |
|             | Notations & Symbols        | xiii     |
| Chapter 1   | Introduction               | 1-6      |
|             | 1.1 Introduction           | 1        |
|             | 1.2 Literature Review      | 1        |
|             | 1.3 Objectives             | 2        |
|             | 1.4 Significance           | 3        |
|             | 1.5 Research Design        | 3        |
|             | 1.6 Source of Data         | 4        |
|             | 1.7 Chapter Scheme         | 5        |
| Chapter 2   | Requirements Specification | 7-12     |

|           |                                                                                      |       |
|-----------|--------------------------------------------------------------------------------------|-------|
|           | 2.1 User Characteristics                                                             | 7     |
|           | 2.2 Functional Requirements                                                          | 7     |
|           | 2.3 Dependencies                                                                     | 9     |
|           | 2.4 Performance Requirements                                                         | 10    |
|           | 2.5 Hardware Requirements                                                            | 10    |
|           | 2.6 Constraints & Assumptions                                                        | 11    |
| Chapter 3 | Design                                                                               | 13-26 |
|           | 3.1 Algorithm                                                                        | 13    |
|           | 3.2 Function Oriented Design for procedural approach                                 | 14    |
|           | 3.3 System Design                                                                    | 15    |
|           | 3.3.1 Data Flow Diagrams                                                             | 16    |
|           | 3.3.2 Activity Diagram                                                               | 18    |
|           | 3.3.3 Flow Chart                                                                     | 19    |
|           | 3.3.4 Class Diagram                                                                  | 20    |
|           | 3.3.5 ER Diagram                                                                     | 21    |
|           | 3.3.6 Sequence diagram                                                               | 22    |
|           | 3.4 Database Design                                                                  | 23    |
|           | 3.4.1 Logical Database Design                                                        | 23    |
|           | 3.4.2 Physical Database Design                                                       | 24    |
| Chapter 4 | Implementation, Testing, and Maintenance                                             | 27-34 |
|           | 4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation | 27    |
|           | 4.2 Testing Techniques and Test Plans (According to project)                         | 28    |

|           |                                                         |       |
|-----------|---------------------------------------------------------|-------|
|           | 4.3 Installation Instructions                           | 30    |
|           | 4.4 End User Instructions                               | 32    |
| Chapter 5 | Results and Discussions                                 | 35-45 |
|           | 5.1 User Interface Representation                       | 35    |
|           | 5.2 Brief Description of Various Modules of the system  | 36    |
|           | 5.3 Snapshots of system with brief detail of each       | 37    |
|           | 5.4 Back Ends Representation                            | 42    |
|           | 5.5 Snapshots of Database Tables with brief description | 44    |
| Chapter 6 | Summary and Conclusions                                 | 46-48 |
| Chapter 7 | Future scope                                            | 49-52 |
|           | Appendix                                                | 53    |
|           | Bibliography                                            | 55    |

## List of Figures

| Figure No. | Figure Title                | Page No. |
|------------|-----------------------------|----------|
| Figure 3.1 | Data Flow Diagram (Level 0) | 16       |
| Figure 3.2 | Data Flow Diagram (Level 1) | 17       |
| Figure 3.3 | Activity Diagram            | 18       |
| Figure 3.4 | Flow Chart                  | 19       |
| Figure 3.5 | Class Diagram               | 20       |
| Figure 3.6 | ER Diagram                  | 21       |
| Figure 3.7 | Sequence Diagram            | 22       |
| Figure 5.1 | Home Page                   | 38       |
| Figure 5.2 | Login Page                  | 39       |
| Figure 5.3 | Generator Page              | 40       |
| Figure 5.4 | Generated Output Screen     | 41       |
| Figure 5.5 | User Table (Database)       | 44       |
| Figure 5.6 | Content Table (Database)    | 45       |

## List of Tables

| Table No. | Table Title                     | Page No. |
|-----------|---------------------------------|----------|
| Table 3.1 | User Table (Logical Design)     | 23       |
| Table 3.2 | Content Table (Logical Design)  | 23       |
| Table 3.3 | User Table (Physical Design)    | 25       |
| Table 3.4 | Content Table (Physical Design) | 25       |
| Table 4.1 | Test Cases                      | 29       |

## Abbreviations

| Abbreviation | Full Form                                 |
|--------------|-------------------------------------------|
| AI           | Artificial Intelligence                   |
| UGC          | User Generated Content                    |
| API          | Application Programming Interface         |
| UI           | User Interface                            |
| UX           | User Experience                           |
| DBMS         | Database Management System                |
| SQL          | Structured Query Language                 |
| ORM          | Object Relational Mapping                 |
| PERN         | PostgreSQL, Express.js, React.js, Node.js |
| HTTP         | HyperText Transfer Protocol               |
| REST         | Representational State Transfer           |
| JSON         | JavaScript Object Notation                |
| IDE          | Integrated Development Environment        |
| VPS          | Virtual Private Server                    |
| UAT          | User Acceptance Testing                   |
| DFD          | Data Flow Diagram                         |
| ER           | Entity Relationship                       |
| UML          | Unified Modeling Language                 |

## Notations & Symbols

| Symbol / Notation | Meaning / Description                  |
|-------------------|----------------------------------------|
| →                 | Flow of control / direction of process |
| ↔                 | Bidirectional communication            |
| ()                | Process or function                    |
| []                | Input / Output                         |
| { }               | Data storage / collection              |
| D                 | Data                                   |
| P                 | Process                                |
| U                 | User                                   |
| DB                | Database                               |
| API               | External service interaction           |
| PK                | Primary Key                            |
| FK                | Foreign Key                            |
| 1:N               | One-to-Many Relationship               |
| N:1               | Many-to-One Relationship               |
| ==                | Equality comparison                    |
| !=                | Not equal                              |
| &&                | Logical AND                            |
|                   | Logical OR                             |

# Chapter 1 – Introduction

## 1.1 Introduction

In today's digital era, marketing and advertising play a crucial role in business growth. Creating high-quality advertisements such as images, videos, captions, and scripts requires time, creativity, and resources. Many startups, small businesses, and content creators face challenges in producing engaging content consistently.

To address this problem, the AI-Powered UGC Ads Video & Image Generator is developed as a full-stack web application that automates the creation of advertisement content using Artificial Intelligence. The system allows users to generate marketing content by simply providing a text prompt or uploading media.

The application uses modern technologies such as React for frontend development, Node.js and Express for backend services, and PostgreSQL for data storage. It integrates with the Google Gemini API to generate intelligent and creative advertisement outputs.

This system helps users save time, reduce cost, and improve productivity by automating the ad creation process efficiently.

## 1.2 Literature Review

AI-based content generation has gained significant attention in recent years due to advancements in large language models and generative AI technologies. These systems enable automatic creation of text, images, and videos, reducing manual effort in digital content production. According to Brown et al. (2020), large-scale transformer models such as GPT demonstrate strong capabilities in generating human-like text across multiple domains [1].

In the field of marketing and advertising, AI-driven tools are increasingly used to generate personalized and engaging content. Kumar et al. (2022) highlight that automated ad generation systems can significantly improve efficiency and reduce production costs for businesses by leveraging machine learning techniques [2]. However, early systems relied on template-based or rule-based approaches, which lacked creativity and adaptability.

Recent developments in multimodal AI models have enabled systems to generate not only text but also images and videos from user prompts. OpenAI and Google have introduced advanced

generative models capable of understanding context and producing high-quality outputs. The Google Gemini model, for instance, integrates text and media generation capabilities, making it suitable for applications such as automated advertisement creation [3].

Modern web applications supporting AI services are commonly built using full-stack architectures such as the MERN or PERN stack. These architectures combine frontend frameworks like React with backend technologies such as Node.js and databases like PostgreSQL, providing scalability and real-time interaction capabilities [4]. Additionally, the use of REST APIs enables seamless communication between system components and external AI services.

Security and authentication have also become critical components of AI-powered platforms. Third-party authentication services, such as Clerk, simplify secure user management while ensuring the protection of user data. According to OWASP guidelines, implementing secure authentication and proper data handling practices is essential for modern web applications [5].

Despite these advancements, several challenges remain in terms of system performance, API dependency, and scalability. Many AI-based systems rely heavily on external APIs, which can introduce latency and reliability issues. This project addresses these challenges by integrating efficient backend processing, structured data storage, and user-friendly interfaces to develop a scalable and reliable AI-powered advertisement generation platform.

### **1.3 Objectives**

The main objectives of this project are:

- To develop a full-stack AI-based web application using modern technologies.
- To automate the generation of user-generated content (UGC) advertisements using AI.
- To generate various types of marketing content such as captions, reviews, scripts, images, and videos.
- To provide secure and reliable user authentication for accessing system features.
- To store and manage user data and generated content using PostgreSQL.
- To deploy a production-ready Software-as-a-Service (SaaS) application.
- To learn and implement PERN stack development (PostgreSQL, Express, React, Node.js).

## 1.4 Significance

The AI-Powered UGC Ads Video & Image Generator is significant as it addresses the growing need for fast, efficient, and cost-effective content creation in digital marketing.

In traditional methods, creating advertisements requires professional skills, time, and financial resources. This project simplifies the process by using Artificial Intelligence to automatically generate high-quality advertisement content such as captions, scripts, images, and videos from simple user inputs.

The system is especially beneficial for:

- Startups and small businesses, which may not have dedicated marketing teams
- Content creators and influencers, who need quick and engaging content
- Digital marketers, who require scalable content generation solutions

This project also demonstrates the practical application of modern technologies like the PERN stack and AI integration (Gemini API) in building real-world SaaS applications.

Additionally, it improves productivity, reduces manual effort, and enables users to focus more on creativity and strategy rather than repetitive tasks.

Overall, the system contributes to the advancement of AI-driven automation in the field of digital marketing and content generation.

## 1.5 Research Design

The research design of this project follows a system development approach that focuses on designing, building, and evaluating a full-stack AI-based application for automated advertisement generation.

The development process is divided into several phases:

- Requirement Analysis:  
The system requirements were identified based on user needs such as generating advertisements, secure login, and content management.

- **System Design:**  
A client-server architecture was designed where the React frontend interacts with the Node.js backend. The backend communicates with the PostgreSQL database and integrates with the Gemini API for AI-based content generation.
- **Implementation:**  
The frontend was developed using React for creating an interactive user interface. The backend was built using Node.js and Express to handle API requests and business logic. PostgreSQL was used to store user data and generated content.
- **AI Integration:**  
Google Gemini API was integrated to generate advertisement content based on user prompts and inputs.
- **Testing:**  
The system was tested for functionality, performance, and error handling to ensure smooth operation.
- **Deployment:**  
The final system was deployed on a VPS server to make it accessible online.

This structured approach ensures that the system is reliable, scalable, and meets user requirements effectively.

## **1.6 Source of Data**

The system uses both user-provided data and AI-generated data for its operation.

- **User Input Data:**  
The primary source of data is the input provided by users in the form of text prompts or uploaded media (images). These inputs are used to generate advertisement content.
- **AI-Generated Data:**  
The system uses the Google Gemini API to generate advertisement content such as captions, scripts, images, and videos based on user input.

- **User Account Data:**  
Basic user information such as email and login credentials is collected through the authentication system (Clerk) and used for secure access and personalization.
- **Database Data:**  
All generated advertisements and user-related data are stored in a PostgreSQL database. This includes content history, user preferences, and generated outputs.
- **System Logs and Metadata:**  
Additional data such as request logs, timestamps, and processing details may be stored for performance monitoring and debugging purposes.

The system does not rely on external static datasets, as content is dynamically generated using AI based on user inputs.

## **1.7 Chapter Scheme**

The project report is organized into the following chapters:

- **Chapter 1: Introduction**  
This chapter provides an overview of the project, including background, objectives, significance, research design, and data sources.
- **Chapter 2: Requirements Specification**  
This chapter describes the functional and non-functional requirements of the system, along with user characteristics, constraints, and assumptions.
- **Chapter 3: Design**  
This chapter explains the system architecture and design aspects, including diagrams such as Data Flow Diagram (DFD), Activity Diagram, Class Diagram, ER Diagram, and Sequence Diagram.
- **Chapter 4: Implementation, Testing, and Maintenance**  
This chapter discusses the technologies used for implementation, testing strategies, installation process, and user instructions.

- Chapter 5: Results and Discussion

This chapter presents the system outputs, user interface, module descriptions, and snapshots of the application and database.

- Chapter 6: Summary and Conclusion

This chapter summarizes the overall work done, achievements, and key outcomes of the project.

- Chapter 7: Future Scope

This chapter discusses possible improvements and enhancements that can be made in the future.

# Chapter 2 – Requirements Specification

## 2.1 User Characteristics

The system is designed to support multiple categories of users with different roles, responsibilities, and technical expertise levels. Understanding user characteristics helps in designing an efficient and user-friendly system.

- **Registered Users (Content Creators / Businesses):**

These are the primary users of the system. They interact directly with the application to generate AI-based advertisements, view results, and manage their content history. These users typically have basic to intermediate knowledge of computers and web applications. They expect a simple, intuitive interface with minimal steps to perform tasks. Their main requirement is fast and high-quality content generation.

- **Administrators:**

Administrators are responsible for managing and monitoring the overall system. Their tasks include supervising user activity, ensuring system performance, and handling access control or subscription management. Administrators require moderate to advanced technical knowledge and access to additional system controls. They play a key role in maintaining system reliability and security.

- **Developers / Testers:**

Developers and testers are responsible for the design, implementation, testing, and maintenance of the system. They ensure that the application functions correctly, efficiently, and securely. These users require strong technical knowledge of web development technologies such as React, Node.js, Express, PostgreSQL, and API integration. They are also responsible for debugging issues and improving system performance.

## 2.2 Functional Requirements

Functional requirements define the core functionalities that the system must perform to meet user needs. The following requirements describe how the system should behave in different scenarios:

- **User Registration and Login:**  
The system must allow new users to register using valid credentials and enable existing users to log in securely. Only authenticated users should be able to access protected features.
- **Input Handling:**  
The system must allow users to enter text prompts describing the advertisement idea. It should also support optional media uploads such as images to enhance content generation.
- **Input Validation:**  
The system must validate all user inputs and display appropriate error messages for invalid or empty inputs.
- **AI-Based Content Generation:**  
The system must send user inputs to the backend, which will process the request and communicate with the Google Gemini API to generate advertisement content such as captions, scripts, images, and videos.
- **Output Display:**  
The system must display the generated content in a user-friendly format, allowing users to preview the results clearly.
- **Content Storage:**  
The system must store generated advertisements along with user details in the PostgreSQL database for future access.
- **Content Management:**  
The system must allow users to view, manage, and reuse previously generated advertisements through a dashboard interface.
- **Error Handling:**  
The system must handle errors such as API failures or invalid inputs gracefully and provide clear messages with retry options.
- **Access Control:**  
The system must ensure that only authorized users can access their personal data and generated content.

- Session Management:

The system must maintain user sessions after login and allow users to log out securely.

## 2.3 Dependencies

The proper functioning of the system depends on several external services, technologies, and environmental factors. These dependencies directly influence system performance, availability, and reliability.

- Google Gemini API:

The system depends on the Google Gemini API for generating advertisement content such as captions, scripts, images, and videos. Any downtime, rate limits, or delays in the API may affect content generation performance.

- Clerk Authentication Service:

The system relies on Clerk for user authentication, registration, and session management. Secure access to system features depends on the availability and proper functioning of this service.

- Internet Connectivity:

A stable internet connection is required for communication between the frontend, backend, database, and external APIs. Poor connectivity may lead to delays or failed requests.

- Hosting Environment (VPS Server):

The application is deployed on a VPS server (such as Hostinger). System availability and performance depend on server uptime, CPU, RAM, and network resources.

- Database System (PostgreSQL):

The system depends on PostgreSQL for storing user data, generated content, and history. Database performance and availability are critical for data retrieval and storage operations.

- Software Frameworks and Libraries:

The system depends on frameworks such as React, Node.js, Express, and ORM tools like Prisma. Any compatibility issues or updates in these technologies may impact system behavior.

## 2.4 Performance Requirements

Performance requirements define how efficiently the system should operate under different conditions to ensure a smooth and responsive user experience.

- **Response Time:**  
The system should respond to user actions such as page navigation and data retrieval within 2–3 seconds under normal operating conditions.
- **AI Generation Time:**  
The advertisement generation process using the Gemini API should be completed within 5–10 seconds, depending on the complexity of the input and API response time.
- **Concurrent User Handling:**  
The system should support multiple users simultaneously without significant performance degradation or delays.
- **Data Retrieval Speed:**  
User data and previously generated content should be fetched from the database within 2 seconds to ensure a smooth user experience.
- **System Availability:**  
The system should be available 24/7, except during scheduled maintenance or unforeseen technical issues.
- **Scalability:**  
The system should be capable of handling increased user load and data volume without affecting performance.
- **Error Handling and Recovery:**  
In case of delays, failures, or API timeouts, the system must respond gracefully by displaying appropriate messages and allowing users to retry operations.

## 2.5 Hardware Requirements

The system requires suitable hardware on both the user side and server side to ensure efficient performance and smooth operation.

- **User-Side Hardware:**  
Users can access the system using a desktop, laptop, or smartphone. A minimum of a

dual-core processor and 4 GB RAM is recommended. The device should have a modern web browser and a stable internet connection for accessing the application and AI services.

- **Server-Side Hardware:**

The application is hosted on a VPS (Virtual Private Server). The server should have a multi-core processor, at least 8 GB RAM, and sufficient storage to handle the application, database, and generated content. A reliable and high-speed network connection is required to manage user requests and communicate with external APIs.

## **2.6 Constraints & Assumptions**

### **A. Constraints**

- The system depends on the availability and performance of the Google Gemini API for generating advertisement content. Any downtime or delay may affect system functionality.
- API usage limits and rate restrictions may impact the number of requests that can be processed within a given time.
- The application must be compatible with modern web browsers such as Chrome, Edge, and Firefox.
- System performance may be affected by server limitations such as CPU, RAM, and storage capacity on the VPS.
- Secure authentication must be handled through the Clerk service, and any issues with this service may affect user access.

### **B. Assumptions**

- Users will have a stable internet connection while accessing the system.
- Users will have basic knowledge of using web applications and digital platforms.

- External services such as the Gemini API and Clerk will remain available and function correctly.
- The system will initially handle a moderate number of concurrent users; scalability improvements can be implemented if usage increases.
- The hosting server will maintain reliable uptime and sufficient resources for smooth operation.

## Chapter 3 – Design

### 3.1 Algorithm

The following algorithm describes the step-by-step working of the AI Powered UGC Ads Video & Image Generator system:

1. The user opens the web application.
2. The user registers or logs into the system.
3. The user navigates to the **Ad Generator** page.
4. The user enters a text prompt or uploads media (optional).
5. The system validates the input.
6. If the input is invalid:
  - a) The system displays an error message.
  - b) The user is redirected back to the input step.
7. If the input is valid, the request is sent to the backend server.
8. The backend processes the request.
9. The backend forwards the request to the **Google Gemini API**.
10. The AI generates advertisement content (text, image, or video).
11. The backend receives the generated output.
12. The generated content is stored in a **PostgreSQL** database.
13. The system sends the output to the frontend.
14. The frontend displays the generated advertisement to the user.
15. The user can preview, download, or reuse the content.
16. The process ends.

## 3.2 Function-Oriented Design

The system is designed using a function-oriented approach, where the overall system is divided into smaller functional modules. Each module performs a specific task and together they ensure smooth operation of the application.

Main Functional Modules:

- **User Authentication Module:**  
This module handles user registration, login, and logout functionality. It ensures that only authorized users can access system features using secure authentication (Clerk).
- **Input Handling Module:**  
This module accepts user input in the form of text prompts and optional media uploads. It is responsible for collecting and preparing data for processing.
- **Input Validation Module:**  
This module checks whether the user input is valid. It ensures that the prompt is not empty and uploaded files meet required conditions. Invalid inputs are rejected with proper error messages.
- **Processing Module (Backend):**  
This module processes user requests. It receives input from the frontend and prepares it for AI-based content generation.
- **AI Integration Module:**  
This module communicates with the Google Gemini API. It sends user input and receives generated advertisement content such as captions, scripts, images, or videos.
- **Data Storage Module:**  
This module stores user data and generated content in the PostgreSQL database. It ensures proper organization and retrieval of data.
- **Content Management Module:**  
This module allows users to view, manage, and reuse previously generated ads through the dashboard.

- **Output Display Module:**  
This module presents the generated content to the user in a clear and user-friendly format, including preview and download options.
- **Error Handling Module:**  
This module manages system errors such as invalid input, API failure, or server issues. It ensures that users receive proper messages and retry options.

### 3.3 System Design

The AI Powered UGC Ads Video & Image Generator is designed using a Client–Server Architecture, which ensures efficient communication between different components of the system.

In this design, the system is divided into three main layers: Frontend, Backend, and Database, along with integration of external AI services.

- **Frontend (Client Side):**  
The frontend is developed using React (TypeScript). It provides the user interface where users can register, log in, enter prompts, upload media, and view generated advertisements. It communicates with the backend through HTTP/HTTPS requests.
- **Backend (Server Side):**  
The backend is built using Node.js and Express. It handles all business logic, processes user requests, validates inputs, and communicates with the AI service (Google Gemini API). It also manages data flow between the frontend and database.
- **Database (PostgreSQL):**  
The database stores user information and generated advertisement content. It ensures secure storage and efficient retrieval of data using structured queries and ORM tools like Prisma.
- **AI Service Integration (Google Gemini API):**  
The system integrates with the Gemini API to generate advertisement content. The backend sends user input to the API and receives generated outputs, which are then processed and displayed to the user.

- Authentication System (Clerk):

Clerk is used for secure user authentication and session management. It ensures that only authorized users can access system features.

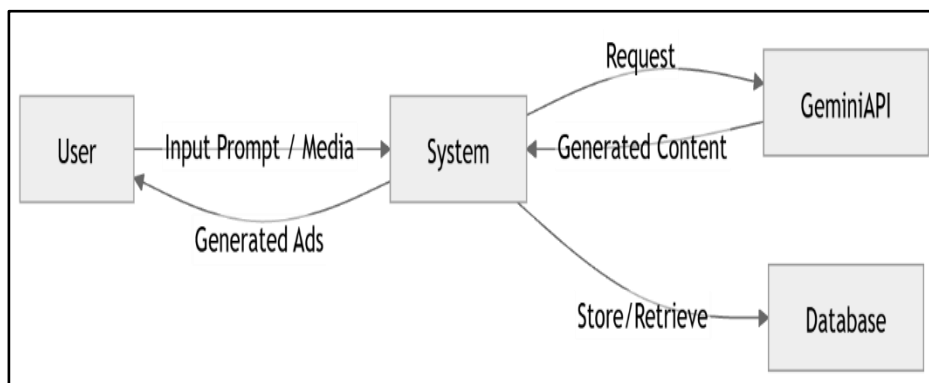
Working Flow of the System:

1. User interacts with the frontend (enters prompt or uploads media)
2. Frontend sends request to backend
3. Backend validates input and processes request
4. Backend calls Gemini API for content generation
5. API returns generated output
6. Backend stores data in database
7. Backend sends result to frontend
8. Frontend displays output to the user

### 3.3.1 Data Flow Diagram

The Data Flow Diagram (DFD) represents how data moves through the system and how different components interact with each other.

A. DFD Level 0

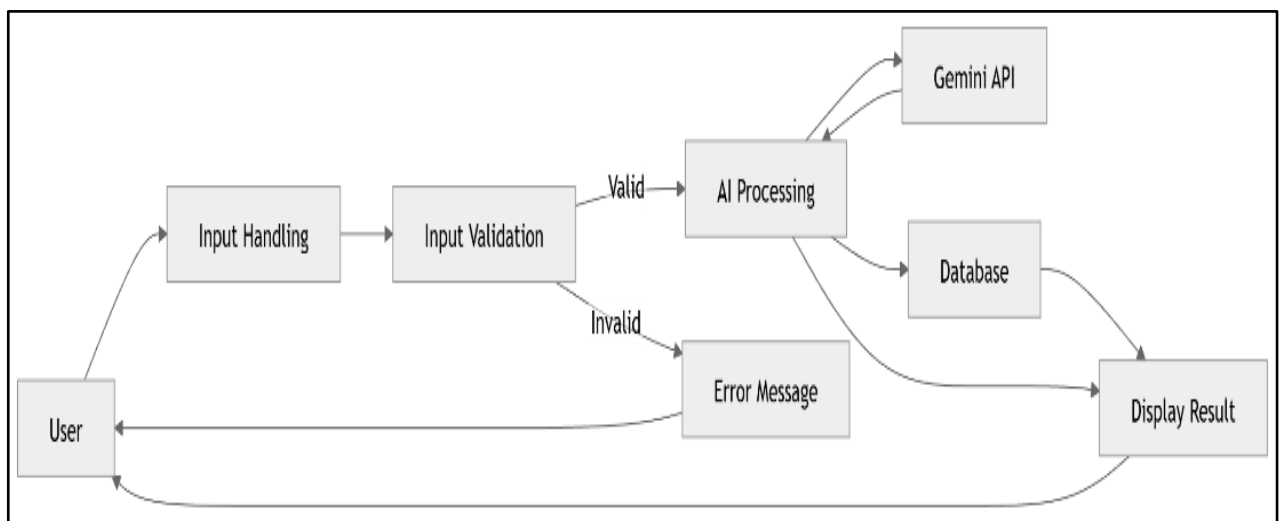


**Figure 3.1: DFD Level 0**

Explanation:

- The User provides input (prompt or media) to the system
- The System processes the request and communicates with the Gemini API
- The generated content is returned to the system and stored in the Database
- The final output is displayed back to the user

## B. DFD Level 1



**Figure 3.2: DFD Level 1**

Explanation:

- The system is divided into multiple processes:
  - **Input Handling:** Accepts user input
  - **Input Validation:** Checks validity of input
  - **AI Processing:** Sends request to AI API
  - **Storage:** Saves generated content
  - **Output Display:** Shows results to user

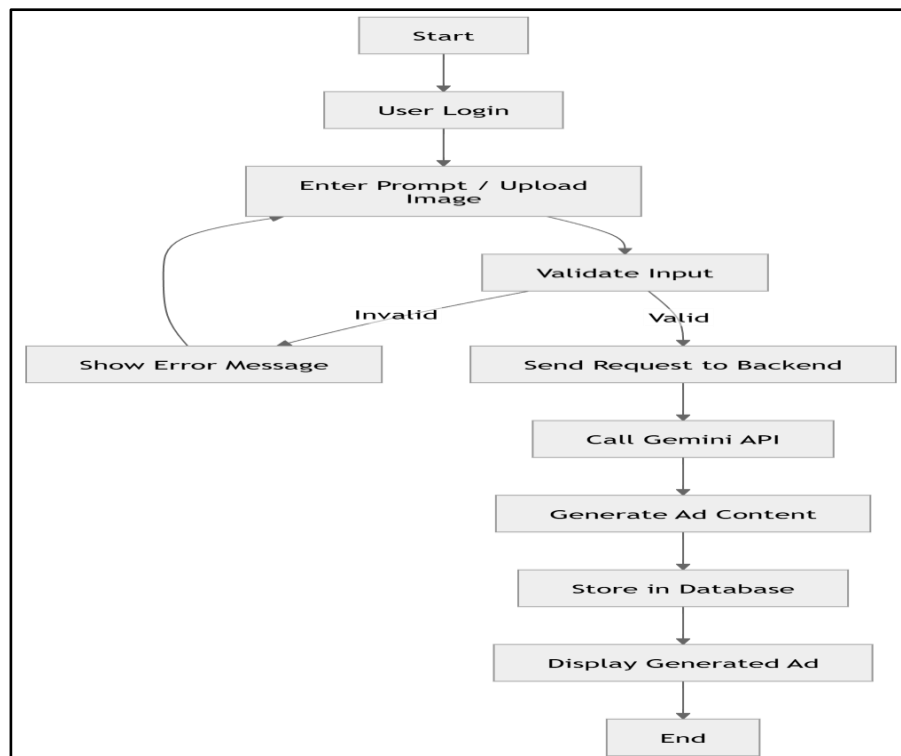
- If input is invalid, an error message is shown
- If valid, the system processes and displays the generated advertisement

### 3.3.2 Activity Diagram

An Activity Diagram represents the workflow of activities performed in the system. It shows the sequence of actions, decision points, and flow of control from start to end.

In the AI Powered UGC Ads Generator, the activity diagram describes how a user interacts with the system to generate advertisements. The process starts when the user enters a prompt or uploads content, followed by input validation. If the input is valid, the request is sent to the backend, which then communicates with the Gemini API to generate advertisement content. The generated output is stored in the database and displayed to the user. If any error occurs, an appropriate message is shown.

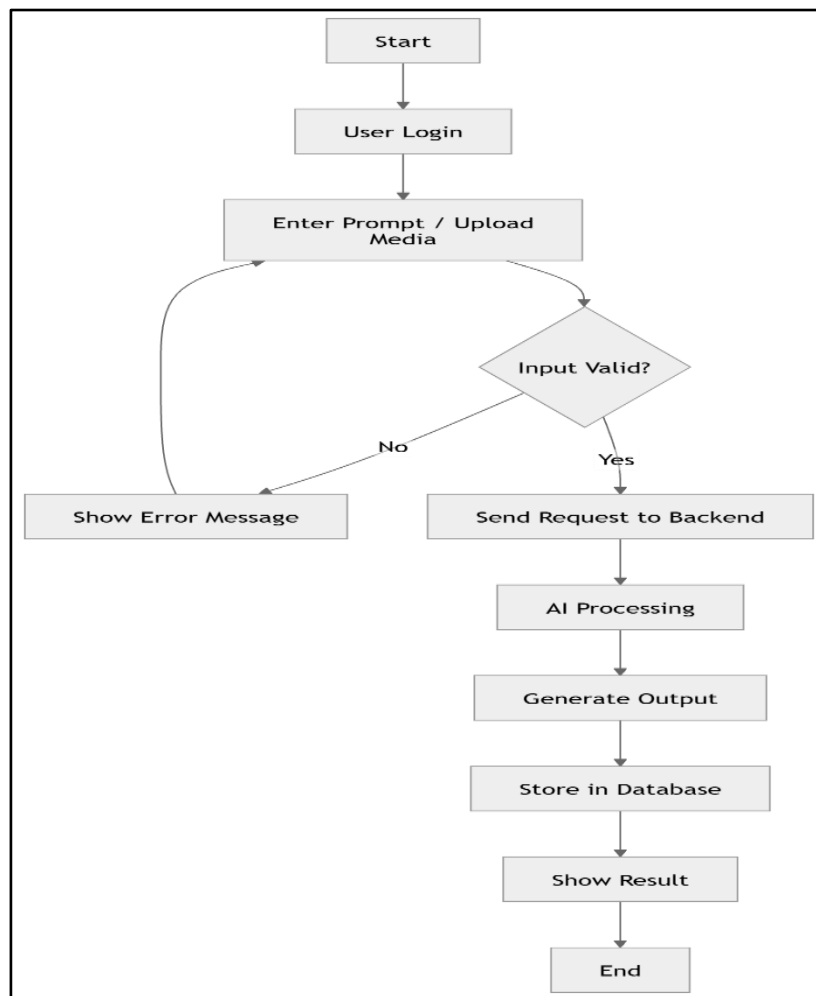
This diagram helps in understanding the complete workflow of ad generation and system behavior.



**Figure 3.3: Activity Diagram**

### 3.3.3 Flow Chart

The flow chart represents the step-by-step execution of the system process, showing the sequence of operations and decision-making involved in generating AI-based advertisements.



**Figure 3.4: Flow Chart**

Explanation:

- The process begins with user login
- The user provides input in the form of a prompt or media
- The system checks whether the input is valid
- If invalid, an error message is shown and the user re-enters input
- If valid, the request is sent to the backend for processing

- The AI generates the output
- The generated content is stored in the database
- Finally, the result is displayed to the user and the process ends

### 3.3.4 Class Diagram

A Class Diagram represents the structure of the system by showing different classes, their attributes, methods, and relationships.

In the AI Powered UGC Ads Generator, the main classes include User, AdContent, and AIService. The User interacts with the system to generate advertisements. Each user can create multiple ad contents. The AdContent class stores details like prompt, generated output, and media. The AIService class handles communication with the Gemini API for content generation.

This diagram helps in understanding system structure and data relationships.

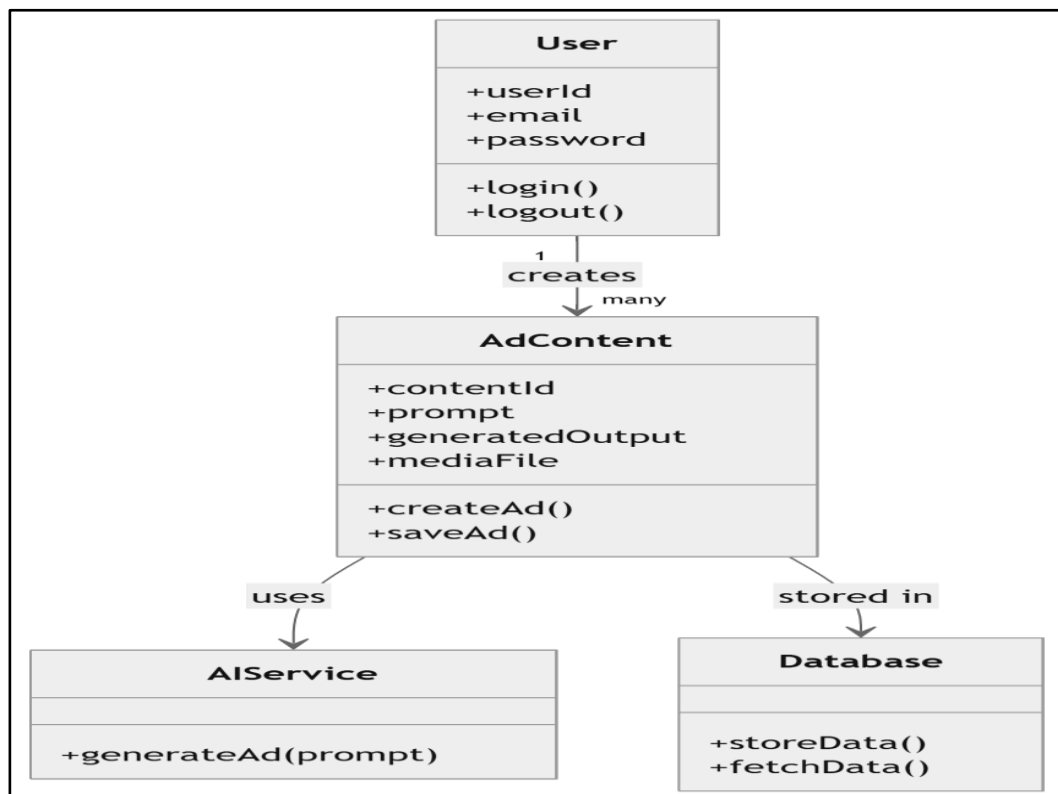


Figure 3.5: Class Diagram

### 3.3.5 ER Diagram

The Entity Relationship (ER) Diagram represents the logical structure of the database, showing entities, their attributes, and relationships between them.

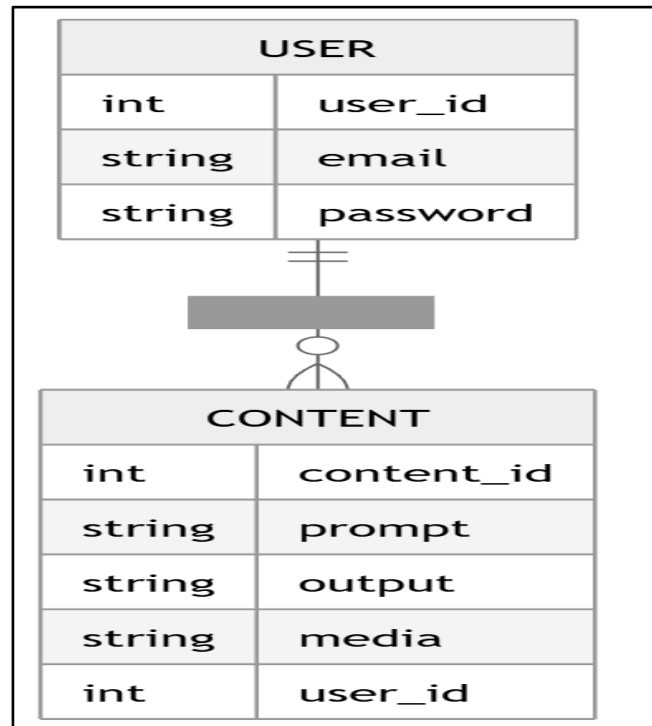


Figure 3.6: ER Diagram

Explanation:

- **USER Entity:**  
Stores user information such as user ID, email, and password.
- **CONTENT Entity:**  
Stores generated advertisement data including prompt, output, and optional media. It also includes a user\_id as a foreign key.
- **Relationship:**
  - One user can generate multiple content entries (one-to-many relationship)
  - Each content is associated with a single user

3.3.6 Sequence Diagram

A Sequence Diagram shows how different components of the system interact with each other in a time sequence.

In this project, it illustrates the interaction between the user, frontend (React), backend (Node.js), Gemini API, and database. The user submits a prompt through the frontend, which sends a request to the backend. The backend processes the request and communicates with the Gemini API to generate content. The result is then stored in the database and returned to the frontend for display.

This diagram helps in understanding the communication flow and system interactions.

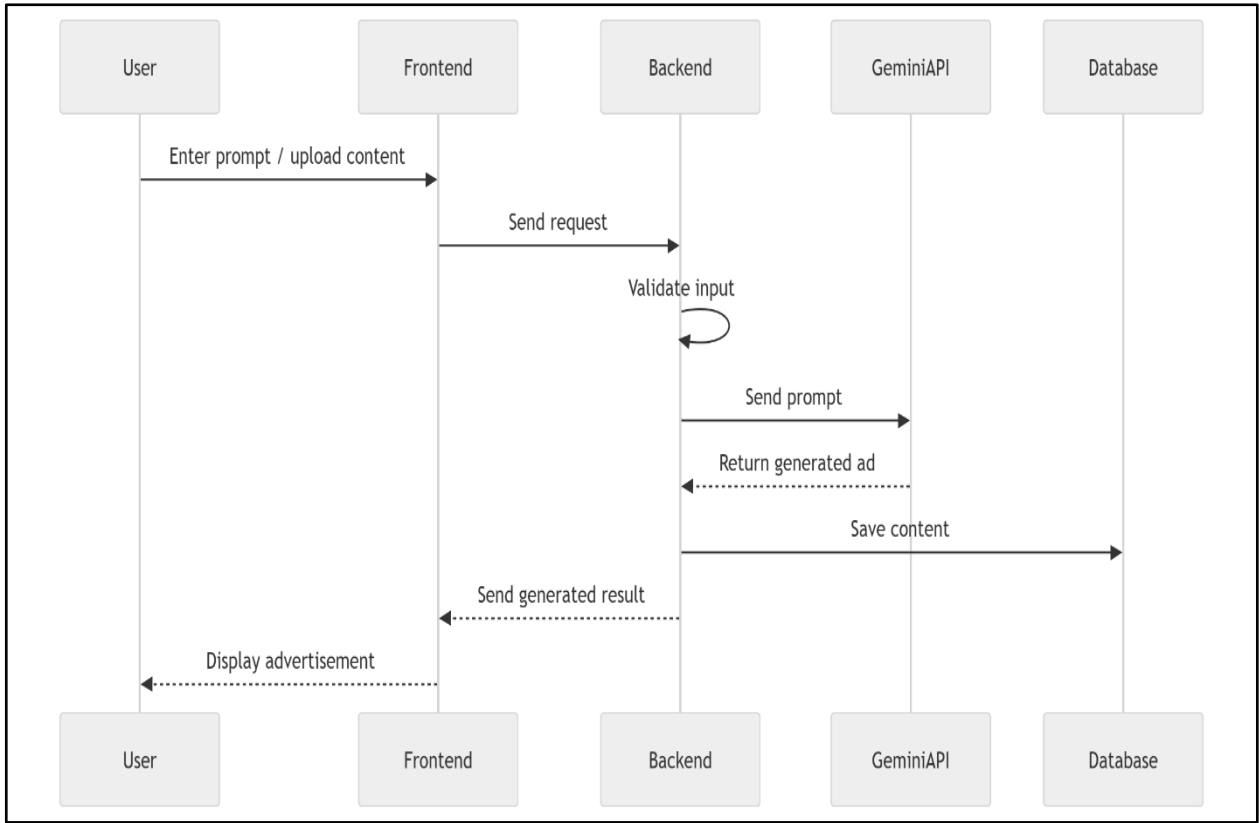


Figure 3.7: Sequence Diagram

## 3.4 Database Design

### 3.4.1 Logical Database Design

The logical database design defines the structure of the database in terms of entities, attributes, and relationships. It ensures proper organization of data for efficient storage and retrieval.

#### A. Database Tables Structure

Table 3.1 – User Table

| Attribute | Data Type | Description                         |
|-----------|-----------|-------------------------------------|
| user_id   | Integer   | Unique ID of the user (Primary Key) |
| email     | String    | User email address                  |
| password  | String    | User password (encrypted)           |

Table 3.2 – Content Table

| Attribute  | Data Type | Description                        |
|------------|-----------|------------------------------------|
| content_id | Integer   | Unique ID of content (Primary Key) |
| prompt     | String    | User input text                    |
| output     | Text      | Generated advertisement content    |
| media      | String    | Uploaded media file (optional)     |
| user_id    | Integer   | Reference to user (Foreign Key)    |

#### B. Relationship

- One User can generate multiple Content records
- Each Content belongs to one User

- Relationship Type: One-to-Many (1: N)

#### C. Constraints

- Primary Key: Ensures uniqueness of records
- Foreign Key: Maintains relationship between User and Content
- Not Null: Important fields (email, password, prompt) cannot be empty

#### D. Design Summary

The logical database design ensures:

- Organized data structure
- Efficient data retrieval
- Strong relationship integrity
- Reduced redundancy

### 3.4.2 Physical Database Design

The physical database design describes how the logical database is implemented in the actual system using a specific database management system. In this project, PostgreSQL (Neon) is used for storing and managing data.

#### A. Implementation Details

- The database is implemented using PostgreSQL, a relational database system
- Tables are created using SQL (Structured Query Language)
- Prisma ORM is used to interact with the database from the backend
- Data is stored in structured format for efficient access and management

## B. Database Tables Structure

Table 3.3 – User Table (Physical Design)

| Column Name | Data Type     | Constraints      |
|-------------|---------------|------------------|
| user_id     | SERIAL        | Primary Key      |
| email       | VARCHAR (255) | NOT NULL, UNIQUE |
| password    | VARCHAR (255) | NOT NULL         |

Table 3.4 – Content Table (Physical Design)

| Column Name | Data Type     | Constraints                          |
|-------------|---------------|--------------------------------------|
| content_id  | SERIAL        | Primary Key                          |
| prompt      | TEXT          | NOT NULL                             |
| output      | TEXT          | NOT NULL                             |
| media       | VARCHAR (255) | NULL (optional)                      |
| user_id     | INTEGER       | Foreign Key REFERENCES User(user_id) |

## C. Additional Design Considerations:

- Indexing:  
Indexes are applied on primary keys and frequently accessed fields to improve query performance
- Security:  
User passwords are stored in encrypted format for security
- Scalability:  
The database structure supports future expansion for additional features such as video data or analytics

- Data Integrity:  
Constraints like Primary Key and Foreign Key ensure consistency and accuracy of data

#### D. Design Summary

The physical database design ensures:

- Efficient data storage and retrieval
- Secure handling of user data
- Strong relational integrity
- High performance using indexing and optimized queries

# Chapter 4 – Implementation, Testing, and Maintenance

## 4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation

The development of the AI Powered UGC Ads Video & Image Generator is carried out using modern technologies and tools to ensure efficiency, scalability, and maintainability of the system.

### 1. Programming Languages:

- JavaScript / TypeScript:  
Used for developing both frontend and backend logic of the application.  
TypeScript improves code reliability with static typing.
- SQL:  
Used for managing and querying the PostgreSQL database.

### 2. Frontend Technologies:

- React.js:  
Used to build dynamic and interactive user interfaces.
- HTML & CSS:  
Used for structuring and styling the web pages.

### 3. Backend Technologies:

- Node.js:  
Provides the runtime environment for executing server-side code.
- Express.js:  
Used to build RESTful APIs and handle client-server communication.

### 4. Database Technologies:

- PostgreSQL (Neon):  
A relational database used to store user data and generated content.

- Prisma ORM:  
Used to interact with the database efficiently using object-oriented queries.

#### 5. AI Integration:

- Google Gemini API:  
Used for generating advertisement content such as captions, scripts, images, and videos based on user input.

#### 6. Authentication:

- Clerk:  
Provides secure user authentication, login, registration, and session management.

#### 7. Tools and IDEs:

- Visual Studio Code (VS Code):  
Used as the primary code editor for development.
- Git & GitHub:  
Used for version control and project management.
- Postman:  
Used for testing backend APIs.

#### 8. Deployment:

- Hostinger VPS:  
Used to host the application and make it accessible online.

## 4.2 Testing Techniques and Test Plans

Testing is an important phase in the development process to ensure that the system functions correctly, efficiently, and without errors. Different testing techniques are used to validate system performance and reliability.

### A. Testing Techniques

- **Unit Testing:**  
Individual components such as input validation, API calls, and database functions are tested separately to ensure they work correctly.
- **Integration Testing:**  
The interaction between frontend, backend, database, and AI API is tested to ensure smooth data flow across modules.
- **System Testing:**  
The complete system is tested as a whole to verify that all functionalities work as expected.
- **User Acceptance Testing (UAT):**  
The system is tested by end users to ensure it meets their requirements and provides a good user experience.
- **Performance Testing:**  
The system is tested under different loads to check response time, AI generation time, and handling of multiple users.

## B. Test Plan

Table 4.1 – Test Cases

| Test Case ID | Description   | Input              | Expected Output               |
|--------------|---------------|--------------------|-------------------------------|
| TC-1         | User Login    | Valid credentials  | Login successful              |
| TC-2         | Invalid Login | Wrong password     | Error message displayed       |
| TC-3         | Generate Ad   | Valid prompt       | Ad generated successfully     |
| TC-4         | Empty Input   | Blank prompt       | Validation error shown        |
| TC-5         | API Failure   | API not responding | Retry/error message displayed |
| TC-6         | View History  | User dashboard     | Previous ads displayed        |

## C. Testing Summary

The testing process ensures:

- System reliability and accuracy
- Proper handling of errors
- Smooth integration between components
- Good user experience

## 4.3 Installation Instructions

This section provides detailed steps to install, configure, and run the AI Powered UGC Ads Video & Image Generator system on a local machine.

### 1. System Requirements

Before installation, ensure the following software and tools are installed:

- Operating System: Windows / Linux / macOS
- Node.js: Version 16 or above
- npm (Node Package Manager)
- PostgreSQL Database (Neon or local setup)
- Git: For version control
- Code Editor: Visual Studio Code (recommended)
- Web Browser: Chrome / Edge / Firefox

### 2. Project Setup

The following steps describe the procedure to set up the project environment:

- Clone the Repository  
Open a terminal or command prompt and execute the following command: `git clone <repository URL>`

This command downloads the project files to the local system.

- Navigate to the Project Directory

Move to the project folder using the following command: `cd project-folder-name`

- Open in Code Editor

Open the project folder in a code editor such as Visual Studio Code for development and further configuration.

### 3. Install Dependencies

- Install backend and frontend dependencies: `npm install`
- This installs all required libraries defined in `package.json`.

### 4. Environment Configuration

Create a `.env` file in the root directory and configure the following:

- `DATABASE_URL` → PostgreSQL connection string
- `GEMINI_API_KEY` → API key for Google Gemini
- `CLERK_API_KEY` / `AUTH KEYS` → Authentication credentials

This step is essential for connecting the system with external services.

### 5. Database Setup

- Ensure PostgreSQL database is running
- Run migration commands (if using Prisma):  
`npx prisma migrate dev`
- This creates required tables such as User and Content

### 6. Running the Application

- Start Backend Server: `npm run server`
- Start Frontend Application: `npm start`

- The frontend will run on:

`http://localhost:3000`

## 7. Verification

- Open the browser and check if the application loads correctly
- Register a new user or log in
- Test ad generation functionality
- Verify database entries

## 8. Troubleshooting

- If dependencies fail → Run npm install again
- If database error occurs → Check DATABASE\_URL
- If API fails → Verify Gemini API key
- If login fails → Check Clerk configuration

## 9. Installation Summary

After successful installation:

- The system will be fully functional
- Users can generate ads, store content, and access dashboard features
- All components (Frontend, Backend, Database, AI API) will be connected properly

## 4.4 End User Instructions

This section provides step-by-step instructions for end users to effectively use the AI Powered UGC Ads Video & Image Generator system.

### 1. Accessing the System

- Open a web browser (Chrome, Edge, or Firefox)
- Enter the application URL (e.g., <http://localhost:3000> or deployed link)
- The Home Page of the application will be displayed

## 2. User Registration

- Click on the Sign-Up button
- Enter required details such as email and password
- Submit the form
- Account will be created successfully

## 3. User Login

- Click on the Login button
- Enter registered email and password
- Click Login
- User will be redirected to the dashboard

## 4. Generating Advertisements

- Navigate to the Generator Page
- Enter a text prompt describing the advertisement  
(Example: “Create a modern ad for a fitness product”)
- Optionally upload an image (if supported)
- Click on the Generate button
- Wait while the system processes the request

## 5. Viewing Generated Content

- The generated advertisement (text/image/video) will be displayed on the screen
- Users can preview the output clearly

- Options may be available to download or reuse the content

## 6. Managing Content (Dashboard)

- Navigate to the Dashboard (MyGenerations)
- View previously generated ads
- Click on any item to preview details
- Reuse prompts for generating new ads

## 7. Exploring Community Content

- Go to the Community Page
- Browse ads created by other users
- Click on content to view details and get inspiration

## 8. Error Handling

- If invalid input is provided → system shows validation message
- If AI processing fails → retry option is displayed
- If login fails → error message is shown

## 9. Logging Out

- Click on the Logout button
- User session will be ended securely

## 10. End User Summary

The system is designed to be:

- Easy to use
- Fast and responsive
- Accessible to users with basic computer knowledge

Users can easily generate, manage, and explore advertisement content using AI with minimal effort.

## Chapter 5 – Results and Discussions

This chapter presents the results obtained from the implementation of the AI Powered UGC Ads Video & Image Generator system. It provides a detailed overview of the user interface, describes the functionality of various system modules, includes system snapshots, and explains the backend database structure used for data storage and management.

### 5.1 User Interface Representation

The user interface of the AI Powered UGC Ads Video & Image Generator is designed to be intuitive, responsive, and easy to use. It enables users to interact with the system efficiently with minimal technical knowledge.

The interface is developed using modern frontend technologies to ensure smooth navigation, fast performance, and a visually appealing layout.

#### A. Key User Interface Screens:

- **Home Page:**  
The home page serves as the entry point of the application. It provides an overview of the platform, its features, and navigation options such as Login, Generator, and Community. It also includes call-to-action buttons like “Get Started”.
- **Login / Signup Page:**  
This page allows users to securely register and log into the system. It ensures authentication before accessing system features.
- **Generator Page:**  
This is the core interface where users can enter text prompts or upload media to generate advertisements. It includes input fields, customization options, and a “Generate” button to initiate AI processing.
- **Output Display Page:**  
After processing, the generated advertisement (image/video/text) is displayed with preview options. Users can view, download, or reuse the generated content.

- **Dashboard (My Generations):**  
This section displays the history of all generated ads. Users can manage their content, view previous outputs, and reuse prompts.
- **Community Page:**  
This page allows users to explore ads created by others. It promotes creativity and provides inspiration through shared content.

## B. Interface Summary

The user interface ensures:

- Easy navigation between different sections
- Minimal user effort for generating ads
- Clear display of results and history
- Responsive design across devices

## 5.2 Brief Description of Various Modules of the System

The AI Powered UGC Ads Video & Image Generator system is divided into multiple modules, each responsible for a specific functionality. This modular design ensures better organization, scalability, and maintainability of the system.

### 1. Authentication Module

This module handles user registration, login, and logout functionalities. It ensures that only authorized users can access the system. Secure authentication and session management are implemented using Clerk.

### 2. Ad Generation Module

This is the core module of the system. It processes user input such as text prompts or uploaded media and generates advertisement content using the Gemini API. The generated output may include captions, scripts, images, or videos.

### 3. Content Management Module

This module is responsible for storing, retrieving, and managing generated advertisements. It maintains user history and allows users to access their previously generated content.

### 4. User Interface Module

This module provides interaction between the user and the system. It includes all frontend components such as forms, dashboards, and display screens developed using React.

### 5. API Integration Module

This module manages communication between the backend and external services. It sends requests to the AI API and receives generated responses for further processing.

### 6. Database Module

This module handles storage of user data and generated content using PostgreSQL. It ensures data consistency, integrity, and efficient retrieval.

### 7. Error Handling Module

This module manages system errors such as invalid inputs, API failures, and server issues. It ensures that appropriate error messages are displayed and the system remains stable.

### Module Summary

The modular structure of the system ensures:

- Clear separation of functionalities
- Efficient system performance
- Easy maintenance and future scalability

## **5.3 Snapshots of system with brief detail of each**

This section presents the visual representation of the system through various screenshots. These snapshots help in understanding the working and interface of different modules of the application.

## 1. Home Page



**Figure 5.1: Home Page**

- Displays the main landing page of the application
- Provides navigation options such as Login, Generator, and Community
- Includes introductory content and call-to-action buttons

## 2. Login Page

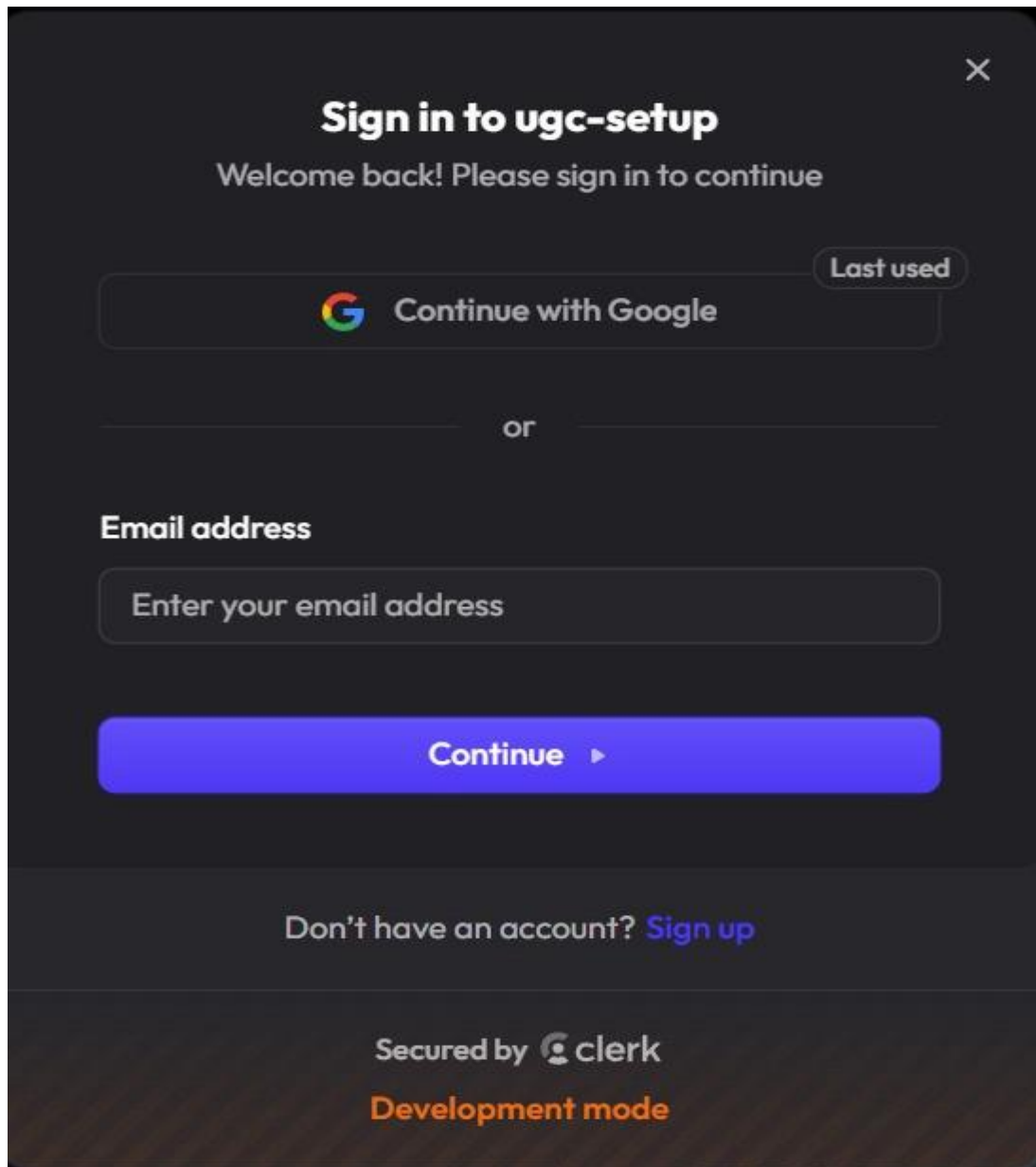


Figure 5.2: Login Page

- Shows the user authentication interface
- Allows users to enter credentials and log into the system
- Displays error messages for invalid login attempts

### 3. Generator Page

The screenshot shows the UGC.AI Generator Page. The header includes the UGC.AI logo, navigation links (Home, Create, Community, Plans), and a Credits: 0 indicator. The main heading is "Create In-Context Image" with a subtext: "Upload your model and product images to generate stunning UGC, short-form videos and social media posts".

On the left, there are two image upload slots. The top slot shows a pink trolley suitcase with the label "Trolley.jpeg". The bottom slot shows a woman in a brown jumpsuit with the label "Chocolate Office Chic ...".

On the right, there are form fields for:

- Project Name: MY FIRST
- Product Name: TROLLY
- Product Description (optional): A young model walking in an airport with a sleek trolley suitcase, travel advertisement, natural pose, soft lighting, realistic shadows, modern branding, high quality, 4k
- Aspect Ratio: Two icons for different aspect ratios are shown.
- User Prompt (optional): Describe how you want the narration to be.

At the bottom, there is a blue button labeled "Generate Image".

Figure 5.3: Generator Page

- Core functionality page of the system
- Allows users to enter prompts or upload media
- Includes “Generate” button to initiate AI processing

#### 4. Generated Output Screen



**Figure 5.4 – Generated Output Screen**

- Displays the advertisement generated by the AI
- Allows users to preview the output clearly
- May include options to download or reuse content

#### Snapshots Summary

The above snapshots demonstrate:

- User-friendly interface design

- Smooth navigation between modules
- Successful generation and display of advertisement content

## 5.4 Backends Representation

The backend of the AI Powered UGC Ads Video & Image Generator is responsible for handling business logic, processing user requests, interacting with external APIs, and managing data storage.

The system uses a PostgreSQL database to store and manage user data and generated advertisement content efficiently.

### 1. Backend Architecture Overview

- The backend is developed using Node.js and Express.js
- It follows a RESTful API architecture for communication between frontend and backend
- The backend processes user inputs and sends requests to the AI (Gemini API)
- The generated results are stored in the database and returned to the frontend

### 2. Database Structure

- The database consists of the following main tables:
- User Table:  
Stores user information such as user ID, email, and password
- Content Table:  
Stores generated advertisement data including prompt, output, media, and user reference

### 3. Database Interaction

1. Prisma ORM is used to interact with the PostgreSQL database
2. It simplifies database operations such as:
  - Insert (saving generated content)
  - Select (retrieving user data and history)
  - Update (modifying records if needed)
  - Delete (removing unwanted data)

#### 4. Data Flow in Backend

1. User sends request from frontend
2. Backend validates the input
3. Request is forwarded to the Gemini API
4. AI generates advertisement content
5. Backend stores the output in database
6. Response is sent back to frontend

#### 5. Backend Summary

The backend ensures:

- Efficient processing of user requests
- Secure storage of user and content data
- Smooth integration with AI services
- Reliable communication between all system components

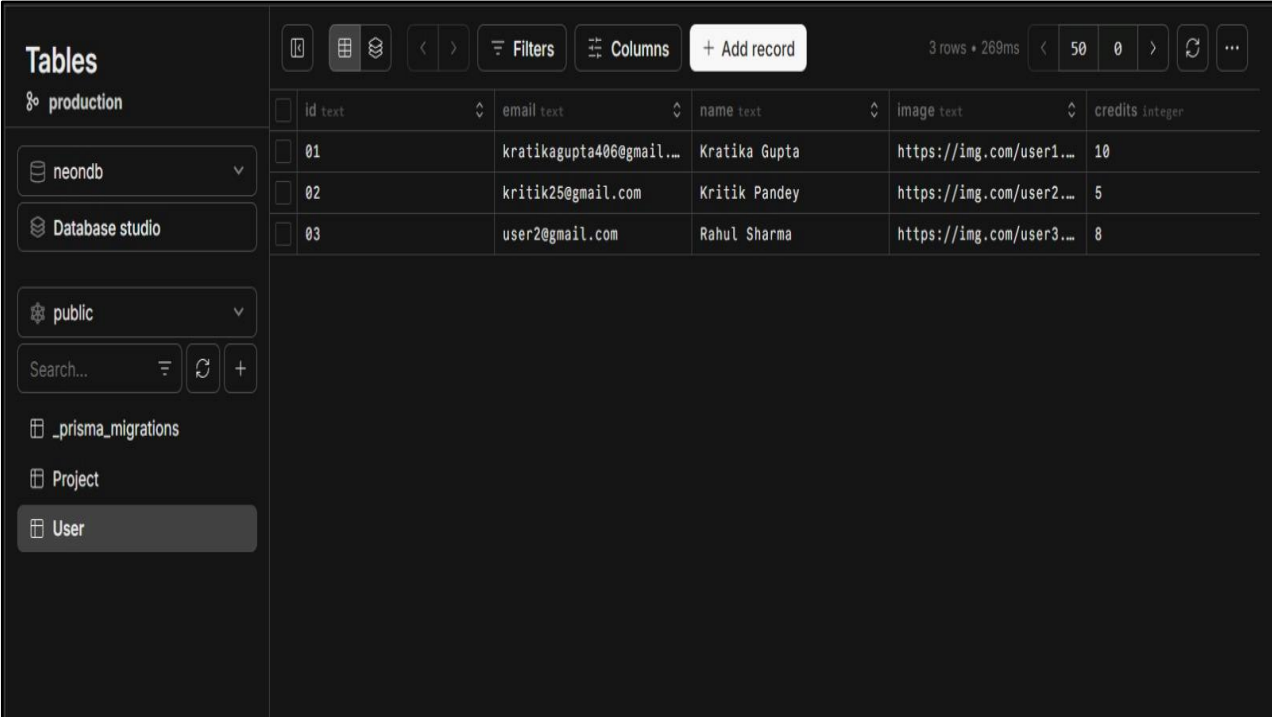
## 5.5 Snapshots of Database Tables with Brief Description

### 1. User Table

The User table stores all registered user details required for authentication and system usage. Each record represents a unique user in the system.

- **id:** Unique identifier assigned to each user
- **email:** Stores the email address used for login
- **name:** Represents the name of the user
- **image:** Contains the profile image URL of the user
- **credits:** Indicates the number of credits available for generating content

The table is used to manage user accounts, track user activity, and control access to system features based on available credits. As shown in the figure, sample user records have been inserted for demonstration and testing purposes.



| id | email                    | name          | image                    | credits |
|----|--------------------------|---------------|--------------------------|---------|
| 01 | kratikagupta406@gmail... | Kratika Gupta | https://img.com/user1... | 10      |
| 02 | kritik25@gmail.com       | Kritik Pandey | https://img.com/user2... | 5       |
| 03 | user2@gmail.com          | Rahul Sharma  | https://img.com/user3... | 8       |

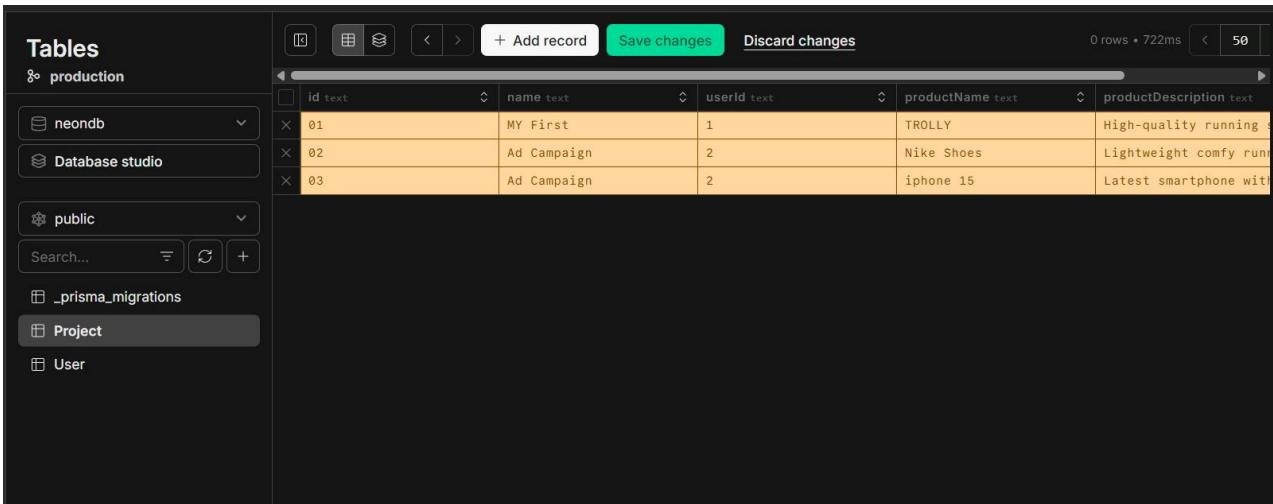
**Figure 5.5 – User Table**

## 2. Content Table

The Content (Project) table stores all advertisement content generated by users. Each entry corresponds to a specific ad campaign created within the system.

- id: Unique identifier for each content record
- name: Name of the advertisement campaign
- userId: Links the content to a specific user
- productName: Name of the product being advertised
- productDescription: Description used to generate advertisement content

This table plays an important role in maintaining generated outputs and allows users to view, edit, and manage their advertisement campaigns efficiently.



| id | name        | userId | productName | productDescription     |
|----|-------------|--------|-------------|------------------------|
| 01 | MY First    | 1      | TROLLY      | High-quality running   |
| 02 | Ad Campaign | 2      | Nike Shoes  | Lightweight comfy run  |
| 03 | Ad Campaign | 2      | iphone 15   | Latest smartphone with |

Figure 5.6 – Content Table

# Chapter 6: Summary and Conclusions

## 6.1 Summary

The AI Powered UGC Ads Video & Image Generator System was designed and developed as a modern solution to automate the creation of digital advertising content using artificial intelligence. In today's fast-growing digital marketing industry, businesses require continuous production of engaging visuals and videos. However, traditional content creation methods are often time-consuming, require technical expertise, and involve high production costs. This system addresses these challenges by enabling users to generate professional-quality content simply by entering text prompts.

The system follows a user-friendly workflow where a user first registers and logs into the platform. After authentication, the user is allowed to enter a prompt describing the type of advertisement, product, or concept they want. The backend AI model processes this input and generate corresponding images or videos based on learned patterns and datasets. The generated content is then displayed to the user and stored in the database for future access.

The system is divided into several functional modules:

- **User Management Module:** Handles registration, login, authentication, and profile management. It ensures secure access to the system using encrypted credentials.
- **Content Generation Module:** This is the core module of the system where AI models interpret user prompts and generate UGC-style advertisements in image or video format.
- **Content Storage Module:** Stores all generated outputs along with metadata such as prompt, user ID, timestamp, and output links.
- **Admin Management Module:** Allows administrators to monitor system usage, manage users, and maintain system stability.
- **Logging and Monitoring Module:** Tracks user activities such as login attempts, content generation requests, and system errors for debugging and performance analysis.

A structured database was implemented to support these modules efficiently. The database ensures proper organization of data and maintains relationships between users and generated

content. Tables such as Users, generated\_Content, Project\_Logs, and Admin were created to handle different types of information systematically.

The user interface was designed with simplicity and clarity in mind. It allows users with no technical background to easily interact with the system. Input fields, buttons, and output preview sections are arranged in a clean layout to improve usability and accessibility.

Overall, the system demonstrates how artificial intelligence can significantly reduce the effort required in content creation while maintaining high quality and consistency in output.

## **6.2 Conclusions**

The development of the AI Powered UGC Ads Video & Image Generator System successfully fulfills its primary goal of automating the creation of advertising content using AI technology. The system proves that machine learning and generative models can be effectively applied in the field of digital marketing to enhance productivity and creativity.

One of the major outcomes of this project is the reduction in dependency on manual design tools and professional editors. Users can generate marketing content in a matter of seconds, which would otherwise take hours or even days using traditional methods. This not only saves time but also reduces operational costs significantly.

The system also ensures scalability, meaning it can handle multiple users and content generation requests simultaneously. The structured database design helps maintain data integrity and allows efficient retrieval of past generated content whenever needed.

In addition, the project highlights the importance of AI-driven automation in modern applications. By integrating natural language processing and generative models, the system bridges the gap between user ideas and visual content creation.

However, like any system, there is always scope for improvement. Currently, the system focuses mainly on text-to-image and text-to-video generation. Future enhancements could include features such as:

- Improved realism and higher-resolution output generation
- Voice-based prompt input for hands-free usage

- Advanced editing tools for customizing generated content
- Multilingual prompt support for global users
- Integration with social media platforms for direct posting
- Faster processing using optimized AI models or cloud acceleration

In conclusion, this project demonstrates a successful implementation of AI in a practical, real-world application. It shows how intelligent systems can transform creative industries by making content generation faster, easier, and more accessible to everyone.

## Chapter 7: Future Scope

The AI Powered UGC Ads Video & Image Generator System is a strong foundation for automating digital content creation using artificial intelligence. While the current system successfully generates images and videos from text prompts, the field of generative AI is rapidly evolving. Therefore, there are many opportunities to extend and enhance this system in the future to make it more powerful, scalable, and industry-ready.

### 1. Advanced AI Model Integration

In the future, the system can be upgraded by integrating more powerful and recent generative AI models such as advanced diffusion models and transformer-based video generation models. These models can produce:

- More realistic human faces and environments
- High-resolution cinematic-quality videos
- Better understanding of complex prompts
- Improved consistency across video frames

This upgrade will significantly enhance the quality of UGC (User Generated Content) advertisements, making them suitable for professional marketing campaigns.

### 2. AI-Powered Script and Story Generation

Currently, the system mainly focuses on generating visuals from prompts. In future development, it can be extended to automatically generate:

- Ad scripts
- Storyboards for videos
- Scene-by-scene breakdowns

This will allow users to input only a product name or idea, and the system will generate a complete advertisement flow including script, visuals, and narration structure.

### **3. Voice-Based Interaction System**

To improve accessibility and user experience, voice-based interaction can be added. Users will be able to:

- Speak prompts instead of typing
- Control video generation using voice commands
- Preview outputs through voice feedback

This feature will make the system more intuitive and useful for non-technical users and mobile users.

### **4. Multilingual and Regional Support**

The system can be expanded to support multiple languages such as Hindi, English, and other regional languages. This will help:

- Reach a wider global audience
- Allow users to create ads in their native language
- Improve personalization for regional marketing campaigns

AI translation models can also be integrated to convert prompts and outputs between languages automatically.

### **5. Advanced Video Editing and Customization Tools**

Future versions can include built-in editing features so that users can modify generated content. These may include:

- Cutting and trimming video clips
- Adding background music and voiceovers
- Applying transitions and filters
- Adjusting brightness, contrast, and saturation

- Adding text overlays and branding logos

This will transform the system from a generator into a complete AI-based video production studio.

## **6. Social Media Automation and Marketing Integration**

A. The system can be directly connected with popular social media platforms such as:

- Instagram
- Facebook
- YouTube Shorts
- LinkedIn Ads

B. This will allow users to:

- Generate content and post it directly
- Schedule advertisements automatically
- Track engagement and performance analytics

This feature will be highly useful for digital marketers and businesses.

## **7. Cloud-Based Scalability and Performance Enhancement**

To support large-scale usage, the system can be migrated to cloud platforms like AWS, Azure, or Google Cloud. Benefits include:

- High-speed processing of AI models
- Support for multiple users simultaneously
- Reduced system downtime
- Scalable storage for generated content
- GPU-based acceleration for faster rendering

This will make the system suitable for enterprise-level deployment.

## **8. AI-Based Personalization Engine**

A recommendation system can be added to analyze user behavior and preferences. It can:

- Suggest better prompts based on past usage
- Recommend trending ad styles
- Automatically generate personalized ad templates
- Improve output quality over time using learning algorithms

This will make the system smarter and more user-centric.

## **9. Real-Time Collaboration Features**

Future development can include collaborative tools where multiple users can:

- Work on the same advertisement project
- Share prompts and outputs
- Edit content in real time
- Provide feedback and revisions

This will be useful for marketing teams and agencies.

# Appendix

The appendix section provides additional technical and supporting information that helps in understanding the project implementation in depth.

## 1. System Architecture Diagrams

- Overall system architecture (Frontend, Backend, AI Model, Database)
- Data flow diagram showing user input to output generation process
- Module interaction diagrams

## 2. Database Design Details

- Complete ER diagram (Entity Relationship Diagram)
- Table relationships and foreign key mapping
- Normalization details (1NF, 2NF, 3NF explanation if required)

## 3. Sample UI Screenshots

- Login and registration pages
- Prompt input interface
- Generated image/video output screen
- Admin dashboard interface

## 4. API Documentation

- Endpoint for user authentication
- Endpoint for content generation
- Endpoint for fetching history
- Endpoint for logs and admin control

## 5. Sample Prompts and Outputs

- Example prompts used by users

- Corresponding generated images/videos
- Comparison of different prompt outputs

## 6. Error Handling and Security Measures

- Input validation techniques
- Password encryption methods
- API error handling responses
- Logging of failed requests

## Bibliography

- [1] Brown, T., Mann, B., Ryder, N. et al., “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [2] Kumar, V., Reinartz, W., “Creating Enduring Customer Value,” *Journal of Marketing*, vol. 80, no. 6, pp. 36–68, 2016.
- [3] Goodfellow, I., Bengio, Y., Courville, A., *Deep Learning*, MIT Press, Cambridge, pp. 1–775, 2016.
- [4] Vaswani, A., Shazeer, N., Parmar, N. et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
- [5] Devlin, J., Chang, M. W., Lee, K., Toutanova, K., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of NAACL-HLT*, vol. 1, pp. 4171–4186, 2019.
- [6] Radford, A., Wu, J., Child, R. et al., “Language Models are Unsupervised Multitask Learners,” *OpenAI Technical Report*, vol. 1, pp. 1–24, 2019.
- [7] OpenAI, “GPT-4 Technical Report,” *arXiv preprint*, vol. 1, pp. 1–100, 2023.
- [8] He, K., Zhang, X., Ren, S., Sun, J., “Deep Residual Learning for Image Recognition,” *Proceedings of CVPR*, vol. 1, pp. 770–778, 2016.
- [9] Krizhevsky, A., Sutskever, I., Hinton, G. E., “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105, 2012.
- [10] Dosovitskiy, A., Beyer, L., Kolesnikov, A. et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *International Conference on Learning Representations (ICLR)*, vol. 1, pp. 1–21, 2021.